8 - Lectura y escritura de archivos

Tabla de contenidos

1 Introducción 1
2 Lectura y escritura 2
2.1 Archivos de texto vs archivos binarios 2
2.2 Los tres pasos fundamentales 2
2.3 Leer archivos 2
2.3.a Leer todo el contenido 3
2.3.b Leer línea a línea 4
2.3.c Leer solo algunas líneas 4
2.3.d Cerrar archivo 5
2.4 Escribir archivos 5
2.5 Uso de la sentencia `with` 7
3 Archivos y rutas 8
3.1 Especificación de rutas con `pathlib.Path` 8
3.1.a Por qué no es suficiente con `str` 8
3.1.b La solución 9
3.2 Combinación de rutas con el operador `/` 9
3.3 Rutas absolutas y relativas 10
3.4 Acceso a directorios de interés 10
3.4.a Directorio de trabajo actual 10
3.4.b Directorio personal (*home*) 11
3.5 Utilidades 11
3.5.a Obtención ruta absoluta y normalizada 11
3.5.b Creación de directorios 11
3.5.c Verificación de validez y tipo de rutas 12
4 Apéndice 12
4.1 El arugmento `mode` de `open()` 12
4.1.a Caracteres válidos 12
4.1.b Ejemplos 12

1 Introducción

En nuestros programas de Python utilizamos variables para almacenar datos durante la ejecución. Estos datos pueden estar escritos directamente en el código o ser ingresados por el usuario (por ejemplo, mediante la función `input()`). Por otro lado, cuando necesitamos mostrar una salida en pantalla, usamos la función `print()`.

Sin embargo, el uso exclusivo de `input()` y `print()` para la entrada y salida de datos tiene limitaciones. Por ejemplo:

- Si trabajamos con más de unos pocos datos, o ni siquiera sabemos cuáles datos necesitaremos al momento de ejecutar el programa, no resulta práctico declararlos en el código o ingresarlos manualmente.
- Si el resultado de nuestro programa consiste en una gran cantidad de datos que no pueden analizarse visualmente con rapidez, o si necesitamos reutilizarlos en otro programa o proceso más adelante, se hace necesario almacenarlos de manera persistente.

En resumen, para resolver problemas de mayor complejidad, vamos a necesitar leer y guardar archivos en la computadora.

2 Lectura y escritura

2.1 Archivos de texto vs archivos binarios

En esta sección vamos a aprender a leer y escribir **archivos de texto plano**.

Los archivos de texto plano contienen únicamente **caracteres básicos de texto**, sin ningún tipo de información adicional. Algunos ejemplos son:

- Archivos de texto genéricos con extensión **`.txt`**
- Archivos de código Python con extensión **`.py`**

Estos archivos pueden abrirse sin dificultad en cualquier editor de texto como el Notepad o Positron, y Python puede leer su contenido y tratarlo como cadenas de texto normales (`str`).

En contraste, existen los **archivos binarios**, que contienen secuencias de bits que no se limitan a representar caracteres de texto. Estos pueden almacenar **cualquier tipo de datos**, como imágenes, sonidos, videos, PDFs, ejecutables, etc. Aunque en este apunte nos enfocamos en los archivos de texto plato, muchos de los principios que veremos también se aplican a los archivos binarios.

2.2 Los tres pasos fundamentales

A la hora de trabajar con archivos en Python, normalmente se siguen **tres pasos**:

1. **Abrir** el archivo con la función **`open()`**, que devuelve un objeto de tipo **`TextIOWrapper`**.
2. **Leer o escribir** en el archivo con los métodos **`read()`** o **`write()`** del objeto `TextIOWrapper`.
3. **Cerrar** el archivo con el método **`close()`** para liberar los recursos y asegurarse de que todos los cambios se guarden correctamente.

2.3 Leer archivos

Para abrir un archivo en Python se utiliza la función `open()`, a la cual se le pasa como argumento la ruta del archivo. Esta ruta puede estar representada con una cadena de texto `str` o un objeto `Path` del módulo `pathlib` (que veremos en una sección más adelante). La función `open()` devuelve un objeto de tipo `TextIOWrapper` que representa al archivo abierto y permite interactuar con él.

Supongamos que tenemos un archivo llamado `pensamientos.txt` con el siguiente contenido:

```
Estoy aprendiendo a leer archivos en Python.
No se con qué me voy a encontrar.
Pero acá vamos.
```

y usamos la función `open()` para abrir el archivo.

```
archivo = open("pensamientos.txt")
archivo
```

```
<_io.TextIOWrapper name='pensamientos.txt' mode='r' encoding='cp1252'>
```

Vemos que el objeto devuelto por `open()` no solo incluye el nombre del archivo (`pensamientos.txt`), sino también `mode="r"` y `encoding="cp1252"`.

Tanto `mode` como `encoding` son argumentos de la función `open()`. El primero indica el modo en el que se abre el archivo. Por defecto, este valor es `"r"`, lo que significa que el archivo se abre en modo lectura de texto plano. En este modo es posible leer su contenido, pero no escribir sobre él.

El segundo argumento, `encoding`, especifica la codificación que se usará para convertir los *bytes* del archivo en cadenas de texto de Python. En macOS y Linux el valor por defecto es `"utf-8"`. En cambio, en Windows, la codificación predeterminada es `"cp1252"` (ASCII extendido). Como esto puede generar errores al leer archivos de texto UTF-8 que contengan caracteres no ingleses en Windows, se recomienda siempre incluir explícitamente el argumento `encoding="utf-8"`.

El objeto de la variable `archivo` es de tipo `TextIOWrapper`. A pesar de que el nombre pueda parecer complicado, no es más que otro tipo de objeto en Python, como son las listas o los diccionarios. Cada vez que necesitemos leer o escribir en el archivo, lo haremos a través de los métodos asociados a este objeto.

> ⚠ Puede fallar
>
> Si le pasamos a la función `open()` un nombre de un archivo que no existe, ya sea porque escribimos mal la ruta o cometimos un error de tipeo, obtendremos un `FileNotFoundError`.
>
> ```
> open("pensamiento.txt")
> ```
>
> ```
> FileNotFoundError: [Errno 2] No such file or directory: 'pensamiento.txt'
> ```

2.3.a Leer todo el contenido

Una forma de leer un archivo de texto plano en Python es cargar todo su contenido de una sola vez como una única cadena de texto. Para ello se utiliza el método `.read()` del objeto `TextIOWrapper`.

```
archivo = open("pensamientos.txt", encoding="utf-8")
contenido = archivo.read()
contenido
```

```
'Estoy aprendiendo a leer archivos en Python.\nNo se con qué me voy a
encontrar.\nPero acá vamos.'
```

Se puede observar que, salvo la última, cada línea termina con un carácter de nueva línea (`\n`). Si hubiéramos agregado un salto de línea al final del archivo `pensamientos.txt`, la cadena resultante también habría terminado en `"\n"`.

2.3.b Leer línea a línea

Una alternativa al método `.read()`, que carga todo el contenido como una única cadena de texto, es el método `.readlines()`. Este devuelve una **lista de cadenas**, donde cada elemento corresponde a una línea del archivo.

```
archivo = open("pensamientos.txt", encoding="utf-8")
archivo.readlines()
```

```
['Estoy aprendiendo a leer archivos en Python.\n',
 'No se con qué me voy a encontrar.\n',
 'Pero acá vamos.']
```

Trabajar con una lista de cadenas suele ser más cómodo que manejar un único bloque de texto, ya que te permite acceder directamente a cada línea por separado.

2.3.c Leer solo algunas líneas

También es posible leer un número limitado de líneas en lugar de cargar todo el archivo de una sola vez. Para ello se utiliza el método `.readline()`, en singular. Cada vez que se invoca, se obtiene la siguiente línea del archivo. Cuando ya no quedan más líneas, devuelve una cadena vacía.

```
archivo = open("pensamientos.txt", encoding="utf-8")
while True:
    linea = archivo.readline()
    if linea == "":
        break
    print(linea, end="") # `end=""` para que Python no agregue salto de línea
```

```
Estoy aprendiendo a leer archivos en Python.
No se con qué me voy a encontrar.
Pero acá vamos.
```

Este método también acepta un argumento opcional `size`, que permite especificar la cantidad de caracteres a leer en cada llamada.

2.3.d Cerrar archivo

Cuando terminamos de trabajar con un archivo, es importante cerrarlo con el método `.close()`. Esto libera los recursos asociados y, en caso de escritura, asegura que todo lo que estaba en el búfer se guarde correctamente.

Aunque Python suele cerrar los archivos automáticamente al final del programa, es una buena práctica hacerlo de forma explícita para evitar comportamientos inesperados.

⚠️ Lecturas sucesivas

Cuando se lee un archivo en Python, existe un puntero interno (a veces llamado *posición en el búfer*) que avanza a medida que se consume el contenido. Por eso, después de una primera lectura completa, las siguientes llamadas a métodos como `.read()` o `.readlines()` no devuelven nada: el puntero ya está al final del archivo.

```
archivo = open("pensamientos.txt")
archivo.read()
```

```
'Estoy aprendiendo a leer archivos en Python.\nNo se con qué me voy a
encontrar.\nPero acá vamos.'
```

```
archivo.read()
```

```
''
```

```
archivo.readlines()
```

```
[]
```

Para volver a leer desde el inicio, hay dos opciones:

- Cerrar y volver a abrir el archivo.
- Usar el método `.seek(0)` para mover el puntero de vuelta al principio.

2.4 Escribir archivos

Para escribir un archivo en Python, primero debemos abrirlo con la función `open()`, de la misma manera que al leer. Sin embargo, no podemos usar los argumentos por defecto, ya que en ese caso el archivo se abre en **modo lectura**, lo que impide escribir en él.

Para poder escribir, el archivo debe abrirse en uno de los siguientes modos:

- **Modo escritura (`"w"`)**: sobrescribe por completo el archivo existente, de forma similar a cuando asignamos un nuevo valor a una variable reemplazando el anterior.

- **Modo adición (`"a"`)**: agrega nuevo contenido al final del archivo existente, sin borrar lo que ya contenía.

Si el archivo indicado en **`open()`** no existe, tanto el modo escritura como el modo adición crearán un **archivo nuevo y vacío**.

Debajo, abrimos un archivo llamado `conclusiones.txt` en **modo escritura**. Como el archivo todavía no existe, Python lo crea automáticamente. Luego, llamamos a **`write()`** sobre el archivo abierto y le pasamos la cadena `"Escribir en Python no es tan grave como parece.\n"` y el texto se escribe dentro del archivo.

```
archivo = open("conclusiones.txt", mode="w", encoding="utf-8") # Este paso ya
crea el archivo
archivo.write("Escribir en Python no es tan grave como parece.\n")
```

```
48
```

El método `.write()` devuelve el número de caracteres escritos, incluyendo el salto de línea `\n`. Después, cerramos el archivo.

```
archivo.close()
```

Para **agregar texto sin reemplazar el contenido existente**, abrimos el archivo en **modo adición**. Escribimos la cadena `"Es solo cuestión de practica."` y lo cerramos nuevamente.

```
archivo = open("conclusiones.txt", mode="a", encoding="utf-8")
archivo.write("Es solo cuestión de practica.")
archivo.close()
```

Finalmente, para mostrar en pantalla el contenido de `conclusiones.txt`, abrimos el archivo en modo lectura, cargamos su contenido con `.read()`, lo guardamos en la variable `texto`, cerramos el archivo y luego imprimimos `texto`.

```
archivo = open("conclusiones.txt", mode="r", encoding="utf-8")
texto = archivo.read()
archivo.close()
print(texto)
```

```
Escribir en Python no es tan grave como parece.
Es solo cuestión de practica
```

> **i** Saltos de línea
>
> Es importante tener presente que el método `.write()` no agrega un salto de línea al final del texto de forma automática, a diferencia de `print()`. Si queremos que el contenido se escriba en una nueva línea dentro del archivo, debemos incluir manualmente el carácter `\n`.

2.5 Uso de la sentencia `with`

Cuando abrimos un archivo con `open()`, debemos cerrarlo después con `.close()`. El problema es que a veces podemos olvidarlo o que el programa falle antes de llegar a esa línea.

Para evitarlo, Python ofrece la sentencia `with`, que se encarga de cerrar el archivo automáticamente al terminar el bloque de código, incluso si ocurre un error en el medio.

Debajo, utilizamos la sentencia `with` para crear un archivo y escribir texto en él.

```
with open("ejemplo.txt", mode="w", encoding="utf-8") as archivo:
    archivo.write("¡Hola, mundo!\n")
print(archivo.closed) # Verificar que el archivo está cerrado
```

```
True
```

Luego, podemos utilizar un patrón similar, pero pasando `mode="r"`, para leer el contenido del archivo.

```
with open("ejemplo.txt", mode="r", encoding="utf-8") as archivo:
    contenido = archivo.read()
print(contenido)
print(archivo.closed)
```

```
¡Hola, mundo!
True
```

> *Context managers*
>
> La sentencia `with` no es exclusiva para leer o escribir archivos: en realidad funciona con un objeto especial llamado ***context manager***.
>
> Un *context manager* se usa en Python para manejar recursos que necesitan ser **adquiridos y luego liberados** de forma segura, como archivos, conexiones de red o bloqueos de concurrencia.
>
> La clave es que `with` define un **bloque de código** dentro del cual el recurso está disponible. Al entrar en el bloque, el recurso se prepara (por ejemplo, se abre un archivo) y, al salir, se libera automáticamente, sin importar si la salida fue normal, si hubo un `return` o si se produjo una excepción.
>
> Esto permite que el código sea más claro y seguro, ya que todo el ciclo de vida del recurso queda encapsulado dentro de ese bloque.

3 Archivos y rutas

Un archivo tiene dos características principales: su **nombre** y su **ruta**.

La **ruta** indica la ubicación exacta del archivo dentro de la computadora y puede incluir directorios, subdirectorios y el nombre del archivo con su **extensión** (por ejemplo, `.txt`, `.csv`, `.py`).

A modo de ejemplo, supongamos que tenemos un archivo con el nombre `reporte.docx`, que se encuentra en la ruta:

```
C:\Users\Tomi\Documentos
```

La parte **C:** de la ruta es la **carpeta raíz** (conocida como *root directory* en inglés), la cual contiene a todas las demás carpetas.

En Windows, la carpeta raíz se llama **C:** y también se le conoce como la **unidad C:**. En macOS y Linux, la carpeta raíz se representa con una simple barra inclinada: /.

Por otro lado, **Users**, **Tomi** y **Documentos** son carpetas, también llamadas **directorios**. Las carpetas pueden contener **archivos** y también **otras carpetas** (llamadas **subcarpetas** o **subdirectorios**).

3.1 Especificación de rutas con `pathlib.Path`

3.1.a Por qué no es suficiente con `str`

Por defecto, Python busca los archivos en el **directorio de trabajo actual** (*current working directory*). Por ejemplo:

```
open("archivo.txt")
```

En este caso, `archivo.txt` debe estar en la misma carpeta desde donde se ejecuta el programa. Si no está allí, hay que indicar su ruta absoluta o relativa.

Aunque se pueden usar cadenas de texto (`str`) para definir rutas, esto puede generar problemas porque los sistemas operativos usan separadores distintos. Mientras que Windows usa la barra inclinada hacia atrás `\`, macOS y Linux usan la barra inclinada hacia delante `/`.

3.1.b La solución

Para evitar errores y escribir código que funcione en cualquier sistema, conviene usar `Path` del módulo `pathlib`, que maneja automáticamente las diferencias entre las formas de especificar rutas entre los distintos sistemas operativos.

```
from pathlib import Path
ruta = Path("Users/Tomi/Documentos/projecto.docx")
print(ruta)
```

En Windows se mostrará como:

```
Users\Tomi\Documentos\project.docx
```

Y en macOS o Linux como:

```
Users/Tomi/Documentos/projecto.docx
```

Si en Windows convertimos al objeto `ruta` a una cadena de caracteres, nos encontraremos con el siguiente resultado:

```
str(ruta)
```

```
'Users\\Tomi\\Documentos\\projecto.docx'
```

Las barras invertidas aparecen como dobles barras invertidas porque en Python cada barra invertida debe **escaparse** con otra barra invertida.

3.2 Combinación de rutas con el operador `/`

El operador `/`, que normalmente usamos para dividir números, también sirve en Python para **combinar rutas** cuando trabajamos con `pathlib.Path`. Esto permite construir rutas de forma clara y sin preocuparse por los separadores que cambian según el sistema operativo. Por ejemplo:

```
from pathlib import Path
ruta = Path("datos") / "usuarios" / "reporte.txt"
print(ruta)
```

El resultado será distinto según el sistema:

En **Windows**:

```
datos\usuarios\reporte.txt
```

En **macOS/Linux**:

```
datos/usuarios/reporte.txt
```

De esta manera, podemos ir "pegando" carpetas y archivos paso a paso, evitando concatenar cadenas manualmente y asegurando compatibilidad multiplataforma.

3.3 Rutas absolutas y relativas

Existen dos formas principales de escribir la ruta de un archivo:

- **Ruta absoluta**: indica la ubicación completa **desde la carpeta raíz**. En Windows comienza con la letra de unidad, por ejemplo `C:\`, y en macOS/Linux comienza con `/`.
- **Ruta relativa**: se interpreta **a partir del directorio de trabajo actual** del programa.

Además, es común usar dos "atajos especiales" dentro de una ruta:

- `.`: indica el directorio actual.
- `..`: indica el directorio padre, del ingles *parent directory*, que es el directorio que contiene al directorio actual.

Por ejemplo, si el directorio de trabajo es `C:\trabajo` y queremos acceder al archivo `C:\trabajo\datos.txt` mediante una ruta relativa, podemos usar simplemente:

```
datos.txt
```

o

```
.\datos.txt
```

Por otro lado, si queremos acceder al archivo `C:\estudio\reporte.txt`, la ruta relativa sería:

```
..\estudio\reporte.txt
```

Esto facilita moverse entre carpetas sin necesidad de escribir rutas absolutas cada vez.

3.4 Acceso a directorios de interés

3.4.a Directorio de trabajo actual

El **directorio de trabajo actual** es la carpeta desde la cual Python busca los archivos cuando usamos rutas relativas. En Python se puede obtener como una cadena de texto usando el método `.cwd()` (del inglés, *current working directory*) de `Path`:

```python
from pathlib import Path
print(Path.cwd())
```

Si necesitamos cambiar el directorio de trabajo actual, se puede usar `chdir` del módulo `os`:

```python
import os
os.chdir("nueva_ruta")
```

> ⚠ Precauciones
>
> Hay que tener en cuenta que al modificar el directorio de trabajo cambian todas las rutas relativas en el programa. Por eso, en proyectos grandes suele ser más seguro usar rutas absolutas o construirlas a partir de un directorio base bien definido.

3.4.b Directorio personal (*home*)

Cada usuario tiene un directorio personal que depende del sistema operativo: en Windows suele estar en `C:\Users\<usuario>`, en macOS en `/Users/<usuario>` y en Linux en `/home/<usuario>`.

En Python se puede acceder a este directorio con el método `Path.home()`:

```python
from pathlib import Path
print(Path.home())
```

Normalmente, los programas tienen permisos de lectura y escritura dentro de este directorio, por lo que es un lugar seguro y conveniente para guardar archivos y configuraciones creadas por tus *scripts*.

3.5 Utilidades

3.5.a Obtención ruta absoluta y normalizada

El método `.resolve()` de `Path` convierte una ruta relativa en absoluta y, cuando es posible, normaliza la ruta colapsando los atajos `.` y `..`. En el siguiente ejemplo, el directorio de trabajo actual es `C:\Users\Tomi\Desktop\proyecto`:

```python
from pathlib import Path

ruta = Path("../datos.txt")
print(ruta.resolve()) # ruta absoluta y normalizada
```

```
C:\Users\Tomi\Desktop\datos.txt
```

3.5.b Creación de directorios

El método `.mkdir()` de los objetos `Path` permite crear un directorio a partir de una ruta.

```python
from pathlib import Path

Path("nueva_carpeta").mkdir(exist_ok=True)
```

Con `exist_ok=True` evitamos errores si la carpeta ya existe.

Para crear todos los directorios en una ruta de múltiples directorios en una sola llamada, se puede añadir `parents=True`:

```python
Path("datos/limpios/2025").mkdir(parents=True, exist_ok=True)
```

3.5.c Verificación de validez y tipo de rutas

Los objetos `Path` permiten comprobar si una ruta existe y qué tipo de recurso representa:

```python
from pathlib import Path

ruta = Path("C:/Users/Tomi/Documentos/projecto.docx")

print(ruta.exists())   # True si existe
print(ruta.is_file())  # True si existe y es archivo
print(ruta.is_dir())   # True si existe y es directorio
```

4 Apéndice

4.1 El arugmento `mode` de `open()`

4.1.a Caracteres válidos

Caracter	Significado
`r`	abrir para lectura (predeterminado)
`w`	abrir para escritura, truncando primero el archivo
`x`	crear un archivo nuevo y abrirlo para escritura
`a`	abrir para escritura, agregando al final del archivo si existe
`b`	modo binario
`t`	modo texto (predeterminado)
`+`	abrir un archivo en disco para actualización (lectura y escritura)

4.1.b Ejemplos

Mo-do	Ejemplo	Descripción
`r`	`open("datos.txt", "r")`	Abre `datos.txt` para leer en modo texto.

Modo	Ejemplo	Descripción
`rb`	`open("imagen.png", "rb")`	Abre `imagen.png` para leer en modo binario (útil para imágenes, PDFs, etc).
`w`	`open("salida.txt", "w")`	Abre `salida.txt` para escritura en texto, truncando el archivo si existe.
`wb`	`open("audio.raw", "wb")`	Abre `audio.raw` para escritura en binario, truncando si existe.
`a`	`open("log.txt", "a")`	Abre `log.txt` para añadir texto al final del archivo.
`ab`	`open("video.mp4", "ab")`	Abre `video.mp4` en binario para añadir datos al final.
`x`	`open("nuevo.txt", "x")`	Crea `nuevo.txt` y lo abre para escritura en texto. Falla si ya existe.
`xb`	`open("nuevo.dat", "xb")`	Crea `nuevo.dat` y lo abre para escritura en binario. Falla si ya existe.
`r+`	`open("datos.txt", "r+")`	Abre `datos.txt` para leer y escribir en texto.
`rb+`	`open("imagen.png", "rb+")`	Abre `imagen.png` para leer y escribir en binario.
`w+`	`open("datos.txt", "w+")`	Abre `datos.txt` para leer y escribir en texto, truncando si existe.
`wb+`	`open("datos.bin", "wb+")`	Abre `datos.bin` para leer y escribir en binario, truncando si existe.